The Agent Contract

Our automated tests drive your agent through one command. Everything they rely on is here.

01 The command

We run `agent` once for every user message.

- The message arrives on standard input. Only the newest message arrives, not the conversation so far.
- Your agent works on the project in `WORKSPACE`, keeps anything it needs in `STATE_DIR`, writes its reply to standard output and exits with code 0.
- Every message starts a fresh process, so anything you need to remember between messages goes in `STATE_DIR`.
- Clarifying questions are allowed. End your reply with the question; the user's answer arrives as the next message.
- The reply is everything written to standard output before exit. Any exit code other than 0, or a timeout, fails the message. Standard error is not part of the reply; progress text written to standard output is.
- `agent` must run headless. The automated tests never open your editor; our reviewers use it live.
- We run `agent` from your image's working directory (`WORKDIR`), which must be your repository's root. The starter Dockerfile copies your repository to `/agent` and sets `WORKDIR /agent`. The project you work on is separate, in `WORKSPACE`.
- What happens inside is up to you. Subagents, extra model calls and tools are all fine.

02 Environment

VARIABLE	MEANING
`WORKSPACE`	The project, a git repository. Your changes stay between the messages of one session
`STATE_DIR`	Kept between messages in one session. Empty at the start
`SESSION_ID`	The session. A new session gets a fresh `WORKSPACE` and an empty `STATE_DIR`
`MODEL_BASE_URL`	Our OpenAI-compatible model gateway: `https://hs-gateway-1066893259518.us-central1.run.app`
`MODEL_API_KEY`	A capped key that works for all three models
`MODEL_NAME`	`strong`
`MODEL_NAME_FAST`	`fast`
`MODEL_NAME_EMBED`	`embed`
`CONTRACT_VERSION`	`1`
`TURN_TIMEOUT_SEC`	Seconds this message may take (600)

`HTTP_PROXY` , `HTTPS_PROXY`	The web filter, the only way out of the sandbox (see 05 Sandbox)

Read these at runtime and never hard-code a provider or a key. If you also load a `.env` file, variables already set in the environment take priority.

After each message we commit everything in `WORKSPACE` to its git repository (`git add -A` , author `evaluator`). Each message therefore starts with a clean `git status` , and your earlier changes are in the history, not in uncommitted changes. Use the history (`git log` , `git diff HEAD~1`) to see or undo earlier work. Files you leave in `WORKSPACE` are committed too.

03 Models

NAME	MODEL
`strong`	Gemini 4 “Argon”, effort capped at medium
`fast`	Gemini 3.8 Flash, effort capped at medium
`embed`	Gemini Embedding 2, vectors of 3,072 numbers

- The gateway only accepts these three names.
- Chat completions support `messages` , `tools` , `tool_choice` , `stream` , `response_format` (JSON), `temperature` , `top_p` , `max_tokens` , `stop` , `n` , `seed` and `presence_penalty` . `logprobs` isn’t supported. The Responses API (`/v1/responses`) works too.
- `max_tokens` includes the model’s thinking, and `reasoning_effort` is capped at medium.
- Your repository’s `.env` holds your team’s development key: US$50 across all three models, 60 requests per minute, no top-ups.
- You can develop with any provider on your own key, but judging always uses our gateway and our keys.

04 Time

- Each message can take up to 10 minutes. A session can take up to 60 minutes of your agent’s time.
- We record how long each message takes to produce its first output: the time from the start of `agent` to the first byte on standard output. Responsiveness counts.

05 Sandbox

- Linux on x86-64 (`linux/amd64`), Debian 12, with 2 CPUs, 4 GB of memory per session. There is no set disk limit; keep what you write small.
- Outbound traffic is limited to the model gateway and the websites listed in `kit/allowed-sites.txt` in your repository, and only through `HTTP_PROXY` and `HTTPS_PROXY` . A client that ignores those variables reaches nothing, the gateway included. Other model providers’ APIs are blocked. Every request is logged.
- Package registries are blocked during a run, and any package installed or removed is recorded.
- Only write inside `WORKSPACE` , `STATE_DIR` and the system temp folder.
- Any process started during a message must stop when `agent` exits. Anything still running is stopped and recorded.
- No interactive prompts.

- The test projects are private Python codebases you won't see in advance: Python 3.12, dependencies already installed, tests run with `python -m pytest`.

06 What counts as a fix

A fix counts when the hidden tests pass and the change is clean: at most 30 changed lines (added plus removed) in Python files outside `tests/`; no lines removed from existing test files; nothing suppressed (no `noqa`, `type: ignore`, skip or xfail markers, `except: pass`, and no `assert` removed outside `tests/`); and every changed file still parses, with no new import left unused.

07 Your repository

We create a private repository for your team. Submissions close at 12:00 on Sunday 8 November. We take `main` as it stands at that moment, and the repository becomes read-only at 12:02.

You must have a `hacktheburgh.yaml` at the root:

```
contract_version: "1"
dockerfile: Dockerfile    # default
context: .                # default
agent: agent              # default, relative to the repository root
personas:                 # required: the persona you designed for
  - {name: "...", description: "..."}
based_on:                 # every open-source project you started from ([] if none)
  - {name: "...", url: "https://...", changes: "the main changes you made"}
```

Your image holds the agent core, not the editor. It builds from our base image, `ghcr.io/instalily-x-hacktheburgh/hacktheburgh-base:1`, which includes Python 3.12, git, pytest and the projects' packages. Your Dockerfile takes it as `ARG BASE` and builds `FROM ${BASE}`. It must build for `linux/amd64` within 10 minutes, and everything your agent needs must be installed at build time.

On an Apple silicon Mac, `kit check` builds for the Mac's own chip. Check the judged build with `docker build --platform linux/amd64 .` before you submit.

Before judging, we send your built image two short messages with no network and a model address that does not answer. Your agent must exit 0 with a reply to each within 60 seconds, and keep `STATE_DIR` between them; an image that fails this is left out of the automated tests. `kit check` runs the same check.

08 Check your work

The team kit in your repository runs the same environment we judge in:

```
python3 -m venv .venv && .venv/bin/pip install -e ./kit
.venv/bin/python -m evaluator kit check     # the contract, and building your image
.venv/bin/python -m evaluator kit run       # sample conversations against your agent
.venv/bin/python -m evaluator kit balance   # what is left of your development key
```